sql50题

```sql
-- 以下为面试50题
-- 其中重点为：1/2/5/6/7/10/11/12/13/15/17/18/19/22/23/25/31/35/36/40/41/42/45/46
共16题
-- 超级重点 12、17、18、19、22、23、24、25、35、41、46


-- 1. 查询课程编号为“01”的课程比“02”的课程成绩高的所有学生的学号（重点）
select a.s_id, a.s_score, b.s_score from
(select s_id, c_id, s_score from score where c_id = 1) as a
inner JOIN
(select s_id, c_id, s_score from score where c_id = 2) as b
on a.s_id = b.s_id
where a.s_score > b.s_score;


-- 2. 查询平均成绩大于90分的学生的学号和平均成绩
select a.s_id, b.s_name, a.avgs from
(select s_id, avg(s_score) as avgs from score GROUP BY s_id having avg(s_score)
> 90) as a
inner join student as b on a.s_id = b.s_id;



-- 3. 查询所有学生的学号、姓名、选课数、总成绩
select s2.s_id, s2.s_name, count(s1.c_id), sum(s_score)
from score as s1, student as s2
where s1.s_id = s2.s_id
GROUP BY s_id, s_name;

-- 4. 查询姓柳的老师的个数
select count(distinct t_name) from teacher where t_name like '柳%';

-- 5. 查询没学过费兰德老师课程的学生的学号和姓名(重点)

select s_id, s_name from student
where s_id not in(

select s_id from score
where c_id in(

select c_id from course
where t_id =
(select t_id from teacher where t_name = '费兰德')
)
);

-- 6. 查询学过费兰德老师课程的学生的学号和姓名（重点）
select st.s_id, st.s_name, c_name, t.t_name
From student as st
inner join score as s on st.s_id = s.s_id
inner join course as c on s.c_id = c.c_id
inner join teacher as t on t.t_id = c.t_id
```

```sql
where t.t_name = '费兰德'
ORDER By st.s_id;

-- 7. 查询学过编号为1的课程，并且也学过编号为2的课程的学生学号和姓名（重点）
select s_id, s_name from student
where s_id in
(
select a.s_id from
(select s_id from score where c_id = 1) as a
inner join
(select s_id from score where c_id = 2) as b
on a.s_id = b.s_id
)

-- 8. 查询课程编号为2的总成绩
select sum(s_score)
From score where c_id = 2;

select c_id, sum(s_score), avg(s_score), count(c_id) from score
GROUP BY c_id;

-- 9. 查询所有课程成绩小于90分的学号和姓名

-- 思路
-- 1. 查询每个学生成绩小于90分的课程数cnt1
-- 2. 查询每个学生选了几门课cnt2
-- 3. 如果cnt1 = cnt2, 则就是所要的结果

select a.s_id,student.s_name FROM
(
select s_id, count(*) as cnt from score
where s_score < 90
GROUP BY s_id
) as a
inner join
(
select s_id, count(*) as cnt from score
GROUP BY s_id
) as b
on a.s_id = b.s_id
inner join student
on a.s_id = student.s_id
where a.cnt = b.cnt;


-- 10. 查询没有学全所有课程的学生学号和姓名（重点）

-- 1. 所有的课程数 total
-- 2. 每个学生选了的课程数 cnt
-- 3. total = cnt，得到所有课程都学过的学生s_id
-- 4. 与学生表not in

insert into score values(1, 5, 92);
insert into student values(8, '白沉香', '2005-08-08', '女');

-- 写法1(self)：
select student.s_id, student.s_name
from student where s_id not in
```

```
(
select b.s_id from
(
select count(distinct c_id) as total from course
) as a
cross join
(
select s_id, count(c_id) as cnt
From score
group by s_id
) as b
on a.total = b.cnt
)
-- 写法2：
select st.s_id, st.s_name from
student as st LEFT JOIN score as s on st.s_id = s.s_id
GROUP BY st.s_id
Having count(distinct s.c_id) < (select count(distinct c_id) from course);

-- 11. 查询至少有一门课与学号为1的学生所学课程相同的学号和姓名（重点）

-- 思路：
-- 1. 查询学号为1的学生所学课程有哪些
-- 2. 查看其他学生所学课程是否在其中

select s_id, s_name from student
where s_id in
(
select distinct(s_id)
from score as sc where c_id in
(select c_id from score where s_id = 1)
and s_id != 1
ORDER BY s_id
)

-- 写法2（效率更高）：
select a.s_id, a.s_name from student as a
inner join
(
select distinct(s_id)
from score as sc where c_id in
(select c_id from score where s_id = 1)
and s_id != 1
ORDER BY s_id
) as b on a.s_id = b.s_id;


-- 12. 查询和学号为3的学生所学课程完全相同的同学的学号和姓名（超级重点）

-- 思路（有错误）：
-- 1. 查询学号为3的学生所学课程及课程数cnt1
-- 2. 第一步所得结果与score进行左连接，通过课程号
-- 3. 通过s_id进行分组，统计c_id的个数cnt2
-- 4. 如果cnt2等于cnt1，那就是最终所要的学号

select c.s_id, c.s_name from student as c
inner join
(
```

```
select s_id, count(a.c_id) from score as a
select * from score as a
left outer join
(select c_id from score where s_id = 3) as b
on a.c_id = b.c_id
group by s_id having count(a.c_id) =
(select count(c_id) from score where s_id = 3)

)
as d on c.s_id = d.s_id;

-- 12. 查询和学号为3的学生所学课程完全相同的同学的学号和姓名（超级重点）

-- 思路2：
-- 1. 查询学号为3的学生所学课程及课程数cnt1
-- 2. 查询没有选修过以上课程的学生学号
-- 3. 但同时查询选择课程数与3号同学相同的学生
-- 4. 排除第2步的学生后，剩下的学生中一定是包含所要的最终结果的，
--    但该结果中还包含数目不为cnt1的，最后只需要再确定是否包含
--    在第三步中的学生，就得到最终的结果了

select s_id from student
where s_id not in
(
select distinct s_id from score where c_id not in
(select c_id from score where s_id = 3)
)

and s_id  in
(
select s_id from score
where s_id != 3
GROUP BY s_id HAVING count(c_id) =
(select count(c_id) from score where s_id = 3)
)


-- 13. 查询没学过费兰德老师课程的学生的学号和姓名（与第5题相同）
-- 14. 没找到题目
-- 15. 查询两门及其以上不及格课程的同学的学号，姓名及其平均成绩（重点）

-- 方法一：
select st.s_id, st.s_name, avg(sc.s_score) from student as st
inner join score as sc on st.s_id = sc.s_id
where st.s_id in
(
select s_id from score
where s_score < 60
group by s_id HAVING count(c_id) >= 2
)
GROUP BY s_id, s_name

-- 方法二(myself)：
select st.s_id, st.s_name, avg(sc.s_score) from student as st
inner join score as sc on st.s_id = sc.s_id
where sc.s_score < 60
GROUP BY st.s_id HAVING count(sc.c_id) >= 2
```

```sql
-- 方法三(更优写法)
select a.s_id, a.s_name, b.c
from student as a inner join
(
select s_id, avg(s_score) as c from score
where s_score < 60
group by s_id HAVING count(c_id) >= 2
) as b on a.s_id = b.s_id;


-- 16. 检索课程01的分数低于60分，按分数降序排列的学生信息

select st.s_id, st.s_name, b.s_score
from student as st inner join
(
select s_id, s_score from score
where s_score < 90 and c_id=1
)
as b on st.s_id = b.s_id
ORDER BY b.s_score desc


-- 17. 按平均成绩从高到低显示所有学生的所有课程的成绩以及平均成绩(超级重点)
select * from course;
-- 方法一：（显示并不直观）
select a.s_id, a.c_id, a.s_score, b.avg_score
from score as a
inner join
(
select s_id, avg(s_score) as avg_score from score
GROUP BY s_id
) as b on a.s_id = b.s_id
ORDER BY b.avg_score DESC
-------------------------------------------------------------
-- 方法二：
select
s_id
, Max(case when c_id=1 Then s_score else null end) "语文"
, Max(case when c_id=2 Then s_score else null end) "数学"
, Max(case when c_id=3 Then s_score else null end) "英语"
, Max(case when c_id=4 Then s_score else null end) "体育"
, Max(case when c_id=5 Then s_score else null end) "实验"
, avg(s_score) "平均成绩"
from score
GROUP BY s_id
order by avg(s_score) desc

select
s_id
, min(case when c_id=1 Then s_score else null end) "语文"
, MIN(case when c_id=2 Then s_score else null end) "数学"
, min(case when c_id=3 Then s_score else null end) "英语"
, min(case when c_id=4 Then s_score else null end) "体育"
, min(case when c_id=5 Then s_score else null end) "实验"
, avg(s_score) "平均成绩"
from score
GROUP BY s_id
```

```sql
order by avg(s_score) desc



-- 18. 查询各科成绩最高分、最低分和平均分：以如下形式显示：
-- 课程ID，课程name，最高分，最低分，平均分，及格率，中等率，优良率，优秀率
-- 及格为>=60，中等为：70-80，优良为：80-90，优秀为：>=90 (超级重点)

-- 方法一：
select
c.c_id "课程ID"
, c.c_name "课程name"
, count(sc.s_score) "人数"
, max(sc.s_score) "最高分"
, min(sc.s_score) "最低分"
, avg(sc.s_score) "平均分"
, sum(case when s_score>= 60 then 1 else 0 end)/count(*) "及格率"
, sum(case when s_score >= 70 and s_score <80 then 1 else 0 end)/count(*) "中等
率"
, sum(case when s_score >= 80 and s_score <90 then 1 else 0 end)/count(*) "优良
率"
, sum(case when s_score >= 90 then 1 else 0 end)/count(*) "优秀率"
from score sc inner join course c on sc.c_id = c.c_id
GROUP BY sc.c_id;
-- ----------------------------------------------------------------------------
-
-- 方法二：
select
c.c_id "课程ID"
, c.c_name "课程name"
, count(sc.s_score) "人数"
, max(sc.s_score) "最高分"
, min(sc.s_score) "最低分"
, avg(sc.s_score) "平均分"
, avg(case when s_score>= 60 then 1 else 0 end) "及格率"
, avg(case when s_score >= 70 and s_score <80 then 1 else 0 end) "中等率"
, avg(case when s_score >= 80 and s_score <90 then 1 else 0 end) "优良率"
, avg(case when s_score >= 90 then 1 else 0 end) "优秀率"
from score sc inner join course c on sc.c_id = c.c_id
GROUP BY sc.c_id;

-- 测试
select c_id
, SUM(CASE WHEN s_score >= 60 THEN 1 ELSE 0 END)/count(*)
from score
group by c_id


-- 19、按各科成绩进行排序，并显示排名(超级重点row_number)

-- ROW_NUMBER()  连续排名  1 2 3 4
-- RANK()        跳跃排名  1 2 2 4
-- DENSE_RANK()  密集排名  1 2 2 3

set @rownum := 0;
select s_id, s_score
, @rownum := if(@c_id = c_id, @rownum := @rownum + 1, 1) as rank
, @c_id := c_id as c_id
```

```sql
from score
order by c_id, s_score desc;


-- 20. 查询学生的总成绩并进行排名
select s_id 学号, sum(s_score) 总成绩
from score
GROUP BY s_id
order by 总成绩 desc

-- 21. 查询不同老师所教不同课程平均分从高到低显示(不重点)
select t.t_id, t.t_name, avg(sc.s_score) as average
from score as sc
inner join course as c on sc.c_id = c.c_id
inner join teacher as t on t.t_id = c.t_id
group by t.t_id, t.t_name
order by average desc

select  sc.c_id, c.c_name, t.t_id, t.t_name, avg(sc.s_score) as average
from score as sc
inner join course as c on sc.c_id = c.c_id
inner join teacher as t on t.t_id = c.t_id
group by sc.c_id, c.c_name
order by average desc

-- 22. 查询所有课程的成绩第2名到第3名的学生信息及该课程成绩（超级重点）
-- 用到ROW_NUMBER() 函数
select * from
(
select st.*
, sc.c_id
, sc.s_score
, ROW_NUMBER() over(PARTITION by c_id ORDER BY s_score desc) as m
From score as sc inner join student as st on sc.s_id = st.s_id
) as
a where m in(2, 3)

-- 写法2：
set @rownumber = 0;
select y.s_id, st.s_name, y.c_id, y.s_score, rank from
(
select  x.s_id, x.c_id, x.s_score, rank
From
(
select sc.*,
@rownumber:= IF(@cid = sc.c_id, @rownumber:= @rownumber + 1, 1) as rank,
@cid := sc.c_id
from score as sc
order by sc.c_id, sc.s_score desc
) x where rank in(2,3)
) y inner join student as st on y.s_id = st.s_id;



-- 23. 使用分段[100-85],[85-70],[70-60],[<60]来统计各科成绩，
--     分别统计各分数段人数：课程ID和课程名称 (超级重点)

select c.c_id, c.c_name
```

```sql
, count(case when sc.s_score >= 85 and sc.s_score <= 100 then 1 else null end) "[100-85]"
, sum(case when sc.s_score >= 70 and sc.s_score < 85 then 1 else 0 end) "[85-70]"
, sum(case when sc.s_score >= 60 and sc.s_score < 70 then 1 else 0 end) "[70-60]"
, sum(case when sc.s_score < 60 then 1 else 0 end) "[<60]"
from score as sc
inner join course as c
on sc.c_id= c.c_id
GROUP BY c.c_id, c.c_name


-- 24.查询学生平均成绩及其名次(超级重点)
set @rownum = 0;
select a.*
, @rownum:= @rownum + 1 as rank
from
(
select s_id, avg(s_score) as avgs
from score
GROUP BY s_id order by avgs desc
) a;


-- 25.查询各科成绩前三名的记录（不考虑成绩并列情况）(超级重点)
set @rownum = 0;
select c_id, s_id, s_score, rank
from
(
select c_id, s_id, s_score
, @rownum:= if(@c_id = c_id, @rownum + 1, 1) as rank
, @c_id := c_id
from score
order by c_id, s_score desc
) as a
where rank in (1, 2, 3)

-- 稍显冗余的写法
set @rownum = 0;
select b.c_id
,max(case when b.rank = 1 then b.s_score else null end) "第一"
,max(case when b.rank = 2 then b.s_score else null end) "第二"
,max(case when b.rank = 3 then b.s_score else null end) "第三"
from
(
select a.c_id, a.s_id, a.s_score, a.rank
from
(
select c_id, s_id, s_score
, @rownum:= if(@c_id = c_id, @rownum + 1, 1) as rank
, @c_id := c_id
from score
order by c_id, s_score desc
) as a
where a.rank in (1, 2, 3)
) as b
GROUP BY b.c_id
```

```sql
-- 一步到位 ----------------------------------------------------------
set @rownum = 0;
select c_id
,max(case when rank = 1 then s_score else null end) "第一"
,max(case when rank = 2 then s_score else null end) "第二"
,max(case when rank = 3 then s_score else null end) "第三"
from
(
select c_id, s_id, s_score
, @rownum:= if(@c_id = c_id, @rownum + 1, 1) as rank
, @c_id := c_id
from score
order by c_id, s_score desc
) as a
where rank in (1, 2, 3)
GROUP BY c_id

-- 26. 查询每门课程被选修的学生数

select c_id
, count(s_id)
from score
GROUP BY c_id

-- 27. 查询出只有三门课程的全部学生的学号和姓名

select a.s_id, a.s_name
from student as a
inner join
(
select s_id
from score
group by s_id
having count(c_id)=3
) as b on a.s_id = b.s_id;

-- 28. 查询男生人数、女生人数（重点）
select
count(case when s_gender = '男' then 1 else NULL end) "男生人数"
, count(case when s_gender = '女' then 1 else NULL end) "女生人数"
from student

select
sum(case when s_gender = '男' then 1 else 0 end) "男生人数"
, sum(case when s_gender = '女' then 1 else 0 end) "女生人数"
from student


select s_gender, count(s_id)
from student
group by s_gender;


-- 29. 查询名字中含有"小"字的学生信息

select * from student where s_name like '%三%'; -- 含有
```

```sql
select * from student where s_name like '%三';  -- 以三结尾
select * from student where s_name like '三%';  -- 以三开头

-- 30.没有找到

-- 31. 查询2000年出生的学生名单(重点)

select * from student
where YEAR(s_birth)= '2000';

SELECT year('20020927')
YYYY-MM-DD
YYYYMMDD
YYYY/MM/DD
YYMMDD

-- 32. 查询平均成绩大于等于85的所有学生的学号、姓名和平均成绩
explain
select a.s_id,a.s_name, b.avgs
from student as a
inner join
(
select s_id, avg(s_score) as avgs
from score
GROUP BY s_id HAVING avgs >=85
) as b on a.s_id = b.s_id


-- 33.查询每门课程的平均成绩，结果按平均成绩升序排序，平均成绩相同时，按课程号降序排列
select c_id, avg(s_score) as avgs
from score
GROUP BY c_id
ORDER BY avgs asc, c_id desc

-- 34. 查询课程名称为"数学"，且分数低于60的学生姓名和分数

select st.s_id, st.s_name, b.s_score
from student as st inner join
(
select s_id, s_score
from score
where s_score < 60 and c_id =
(select c_id from course where c_name = '数学')
) as b on st.s_id = b.s_id


-- 35. 查询所有学生的课程及分数情况(超级重点)

-- 学号， 姓名， 语文， 英语， 数学

select sc.s_id, st.s_name
, max(case when c.c_name = '语文' then sc.s_score else null end) '语文'
, max(case when c.c_name = '数学' then sc.s_score else null end) '数学'
, max(case when c.c_name = '英语' then sc.s_score else null end) '英语'
, max(case when c.c_name = '体育' then sc.s_score else null end) '体育'
, max(case when c.c_name = '实验' then sc.s_score else null end) '实验'
from score as sc
inner join student as st on sc.s_id = st.s_id
```

```sql
inner join course as c on sc.c_id = c.c_id
group by sc.s_id


-- 36. 查询任何一门课程成绩在70分以上的姓名、课程名称和分数

select st.s_name, c.c_name, sc.s_score
from student as st
inner join score as sc on st.s_id = sc.s_id
inner join course as c on sc.c_id = c.c_id
where sc.s_score >= 70



-- 37. 查询不及格的课程并按课程号从大到小排列

select sc.s_id, st.s_name, c.c_id, c.c_name, sc.s_score
from score as sc
inner join course as c on sc.c_id = c.c_id
inner join student as st on sc.s_id = st.s_id
where sc.s_score < 60
order by c.c_id desc

-- 38. 查询课程编号为03且课程成绩在80分以上的学生的学号和姓名

select st.s_id, st.s_name, sc.s_score
from student as st
inner join score as sc on st.s_id = sc.s_id
where sc.c_id = 3 and sc.s_score >= 80


-- 39. 求每门课程的学生人数
select c_id, count(s_id)
from score GROUP BY c_id


-- 40. 查询选修“赵无极”老师所授课程的学生中成绩最高的学生姓名及其成绩(重点)


select st.s_id, st.s_name, c.c_name, sc.s_score
from student as st
inner join score as sc on sc.s_id = st.s_id
inner join course as c on sc.c_id = c.c_id
inner join teacher as t on t.t_id = c.t_id
where t.t_name = '赵无极'
order by sc.s_score desc limit 0,1


-- 41. 查询不同课程成绩相同的学生的学生编号、课程编号、学生成绩(重点 -- 有疑问？？)

select  distinct a.s_id, a.c_id, a.s_score
from score as a
cross join score as b
where a.s_score = b.s_score and a.c_id != b.c_id
order by a.s_score desc

-- 42. 查询每门课程成绩最好的前两名 (同22和25题，超级重点)
```

```sql
-- c_id, 第一， 第二

set @rownum = 0;
select a.c_id
, max(case when a.rank = 1 then a.s_score else null end) "第一"
, max(case when a.rank = 2 then a.s_score else null end) "第二"
from
(
select c_id
, s_id
, s_score
, @rownum := IF(@c_id = c_id,@rownum:= @rownum + 1,1) as rank
, @c_id := c_id as c_id2
from score
order by c_id, s_score desc
) as a
GROUP BY a.c_id


-- 43. 统计每门课程的学生选修人数（超过5人的课程才统计）。
--      要求输出课程号和选修人数，查询结果
--      按人数降序排列，若人数相同，按课程号升序排列
select c_id, count(distinct s_id) as cnt
from score
group by c_id having cnt >= 5
order by cnt desc, c_id asc



-- 44. 检索至少选修两门课程的学生学号
select s_id, count(distinct c_id) as cnt
from score
group by s_id having cnt>= 2




-- 45.  查询选修了全部课程的学生信息（重点）
select sc.s_id, count(distinct sc.c_id) as cnt
, st.s_name, st.s_birth, st.s_gender
from score sc
inner join student st on st.s_id = sc.s_id
group by sc.s_id HAVING cnt =
(select count(*) from course)



-- 46. 查询各学生的年龄（重点）
select s_id, s_name
, (Year(CURRENT_DATE()) - Year(s_birth)) as "年龄"
from student

select s_id, s_name, round(DATEDIFF(CURRENT_DATE(),s_birth)/365) as "年龄"
from student



-- 47. 查询没学过“赵无极”老师讲授的任一门课程的学生姓名（同5 重点）
```

```sql
select s_id,s_name from student where s_id not in
(
select distinct s_id from score
where c_id in
(
select c.c_id from course as c
inner join teacher as t on c.t_id = t.t_id
where t.t_name = "赵无极"
)
)



-- 48. 查询两门以上不及格课程的同学的学号及其平均成绩
select s_id, count(s_score), avg(s_score) as cnt
from score where s_score <60
GROUP BY s_id HAVING cnt>=2


-- 49. 查询本月过生日的学生
select * from student
where MONTH(s_birth) = MONTH(NOW())


-- 50. 查询下月过生日的学生
-- a. 基本答案:
select * from student
where MONTH(s_birth) = MONTH(NOW())+1

select month('2021-12-01')+1  -- 13，这里应该为1

select * from student WHERE
CASE WHEN MONTH('2021-12-01') = 12 THEN MONTH(s_birth)=1 ELSE
MONTH(s_birth)=MONTH('2021-12-01') + 1 END

-- b. 完备答案:
select * from student WHERE
CASE WHEN MONTH(NOW()) = 12 THEN MONTH(s_birth)=1 ELSE
MONTH(s_birth)=MONTH(NOW()) + 1 END

-- 51. 查询本周过生日的学生
-- WEEK中第二个参数1的意思是周一作为一周的第一天
select *, WEEK(s_birth,1)  from student

select WEEK(CURRENT_DATE(),1)  -- 本周

-- a. 基本答案:
select * from student
where WEEK(s_birth,1) = WEEK(date(now()),1)


-- 52. 查询下周过生日的学生
-- a.基本答案:
select * from student
where WEEK(s_birth,1) = WEEK(date(now()),1)+1
-- test
```

```
select week('1991-05-20',1) -- 21
select week('2022-05-20',1) -- 20

-- 因为不同年的同一天获取到的week值不一定是相同的
-- 因此在计算时，我们用当前年的年份与生日中的月份拼接后
-- 再来计算week数，就可以解决了
select substring('1990-05-20', 6, 5)
set @yearstr = concat(year(date(now())), '-')
select @yearstr
select concat(@yearstr, substring('1990-05-20', 6,5))
select concat(@yearstr, substring(DATE(NOW()), 6,5))

select week(concat(@yearstr, substring(s_birth, 6,5)))
from student where s_name = '戴沐白'

-- b. 完备答案
set @yearstr = concat(year(date(now())), '-');
select * from student WHERE
week(concat(@yearstr, substring(s_birth, 6,5))) =
week(concat(@yearstr, substring(CURRENT_DATE(), 6,5)))+1

-- 以上答案的bug在于 +1 操作会超出week的最大值范围

select week('2019-12-31',1) -- 53， 如果加1， 就是54了
select week('2020-01-01',1) -- 1


-- c. 最完备答案
set @yearstr = concat(year(date(now())), '-');
select * from student WHERE
CASE WHEN week(concat(@yearstr, substring(CURRENT_DATE(), 6,5)))=53 THEN
week(concat(@yearstr, substring(s_birth, 6,5))) = 1
ELSE
week(concat(@yearstr, substring(s_birth, 6,5))) =
week(concat(@yearstr, substring(CURRENT_DATE(), 6,5)))+1
END
```